Technical Appendix

A Researcher-Centered Workflow for LLM-Assisted Content Analysis

This appendix documents the materials, samples, and analysis rules behind the proposal. Every file it names is in the repository https://github.com/orgsciresearcher-ai/llm-assisted-content-analysis.git, with paths given relative to the repository root. Numbers refer to the first coding run unless a section says otherwise.

A1 Overview of materials

Table A1. Appendix sections and repository files

Section	Topic	Repository files
A2	Prompt architecture with nine modular blocks	`prompts/template/`, `guide/03_prompt.md`, `tools/assemble_prompt.py`
A3	Codebook and code mapping	`codebook/case/`
A4	De-labeled prompt and change log	`prompts/case/`, `tools/build_delabeled_prompt.py`
A5	Model, settings, and output checks	`tools/run_coding.py`, `case/results/first_run.md`
A6	Samples and seeds	`case/samples/`, `tools/draw_samples.py`, `tools/make_blind_sheets.py`
A7	Agreement measures	`tools/agreement.py`
A8	Decision rule of the theory-label test	`case/preregistration/thresholds.md`, `thresholds.json`
A9	Review-effort curve	`tools/review_effort_curve.py`, `guide/07_calibration_and_curve.md`
A10	Precision of the estimates	`guide/07_calibration_and_curve.md`
A11	Recall check of uncoded ads	`case/samples/recall_check_uncoded_ads.csv`
A12	Error modes	`guide/05_review.md`
A13	First-run descriptives	`case/results/first_run.md`, `case/data/`
A14	Second corpus for Phase 2	not yet shared
A15	Costs	`tools/estimate_cost.py`
A16	Data, ethics, and API terms	`case/README.md`, `LICENSE`, `LICENSE-docs.md`
A17	Full coding prompt (prompt A)	`prompts/case/A_system.txt`, `prompts/case/A_user.txt`

A2 Prompt architecture

The prompt template has nine modular blocks and an input slot (Table A2). Block 1 forms the system prompt. Blocks 2 to 9 and the input slot form the user message, which is filled once per document. Each block has one job, so a team can replace the content of a block, such as its own codebook, without touching the others. The template files are in `prompts/template/`. The script `tools/assemble_prompt.py` joins them, removes guidance notes (lines that start with `//`), and stops if a placeholder is left unfilled.

Theory enters the prompt mainly in blocks 2 and 8. The research context names the construct and its theoretical source, and the codebook groups and labels the codes. The system prompt calls the codebook theory-derived, and the construct name and the code prefixes also appear in the rules and in the output schema. The de-labeled prompt changes these passages and nothing else (A4).

Table A2. The nine blocks of the prompt template and the matching parts of the case prompt

#	Block	Purpose	Part of case prompt A (A17)
1	Role statement	role of the model, precision, valid JSON only	system prompt
2	Research context	what the study examines, the code layers	Research context
3	Task	read the whole document, extract and code eligible passages, report documents without codes	Your task
4	Eligibility rule	when a passage may be coded (explicit or contextual reference to the focal topic) and which confidence contextual coding requires	Critical rule: AI specificity requirement
5	Section scope rule	which code layer applies to which section of a document	Critical rule: responsibilities vs. requirements
6	Multiple codes rule	when one passage receives codes from both layers	Critical rule: dual coding GT + SK
7	Primary code rule	when to assign several codes and when only the dominant one	Critical rule: primary code prioritization
8	Codebook	all codes with definition, code-when rule, and indicators	Codebook
9	Output schema and worked examples	JSON fields, output rules, three worked examples	Output format, Examples
	Input slot	document ID, metadata fields, full text	Input

A3 Codebook

The codebook has two layers. Layer 1 holds 17 governance-tension codes in five groups, derived from IS governance theory in the prior study ([Blinded for review]). Layer 2 holds three skill codes based on the skill categories of Todd et al. (1995). Layer 1 codes apply only to passages from responsibility sections and Layer 2 codes only to passages from requirement sections (block 5).

Table A3 lists each code with its definition and code-when rule as worded in prompt A. The indicators, which are example phrases for each code, are in `codebook/case/codebook_A_original.csv`. The second column gives the neutral ID that replaces the code in prompt B, and `codebook/case/mapping_A_B.csv` holds the mapping.

Before the first run, both authors coded 32 calibration ads independently and resolved their disagreements, which sharpened the coding rules. The calibration ads were chosen to cover all seven roles, both languages, short and long texts, and borderline cases. `case/data/calibration_ads.csv` gives the reason for each choice.

Table A3. Codebook with neutral IDs of the de-labeled prompt

Code (A)	ID (B)	Label	Definition	Code when
GT1 Centralization of AI decision rights vs. decentralized experimentation				
GT1.1	L1-01	Enterprise AI ownership & standards	Role given org-wide authority to define AI priorities, standards, policies, or decision rights.	Ad assigns enterprise-level ownership, standard-setting, or governance authority. NOT simple collaboration without authority.
GT1.2	L1-02	Federated coordination across BUs	Role coordinates distributed AI across functions/BUs while preserving local autonomy.	Role integrates, aligns, or harmonizes AI work across units. Code CoE/hub-and-spoke. NOT if confined to one team.
GT1.3	L1-03	Local experimentation enablement	Role enables BU-level experimentation, pilots, sandboxes, or decentralized AI use-case development.	Role explicitly supports local experimentation or bottom-up AI. NOT generic “innovation” alone.
GT2 Innovation and generativity vs. control, risk, and compliance				

Code (A)	ID (B)	Label	Definition	Code when
GT2.1	L1-04	AI opportunity creation & experimentation	Role identifies, initiates, or scales AI opportunities for innovation and value creation.	Ad emphasizes discovery, experimentation, use-case ID, or growth through AI. Exclude pure compliance tasks.
GT2.2	L1-05	Responsible AI, risk & compliance	Role addresses ethical, legal, regulatory, privacy, or risk issues related to AI.	Ad mentions responsible AI, ethics, regulation, legal compliance, or risk mitigation related to AI.
GT2.3	L1-06	Governance frameworks & controls	Role designs or maintains formal governance mechanisms for AI oversight and control.	Role builds frameworks, committees, controls, review processes, or policies. NOT vague “best practices”.
GT2.4	L1-07	AI enablement & change management	Role drives organizational AI adoption by managing the human and cultural side — enablement programs, internal champions, stakeholder engagement, friction reduction.	Ad assigns explicit responsibility for change management, adoption acceleration, workforce enablement, or internal advocacy related to AI. NOT generic “collaboration” without adoption mandate.
GT3 Technical and model-specific expertise vs. enterprise strategic alignment				
GT3.1	L1-08	Deep AI/ML/GenAI expertise	Role requires substantial technical knowledge of AI methods, models, tools, or platforms as a governance dimension.	Ad asks for ML, GenAI, LLMs, data science, AI architecture, or tooling depth. NOT basic digital literacy.
GT3.2	L1-09	AI strategy & business value alignment	Role aligns AI with business strategy, competitive priorities, or value creation.	Ad links AI to enterprise strategy, transformation, or ROI. NOT generic leadership language without strategic alignment.
GT3.3	L1-10	Business-technical translation	Role translates between business needs and AI capabilities, or between executives and technical teams.	Ad explicitly frames role as interpreter, connector, or translator across business and technical domains.
GT4 Data governance vs. model governance & lifecycle accountability				
GT4.1	L1-11	Data governance responsibility	Role responsible for data quality, access, lineage, stewardship, privacy, or governance of inputs to AI.	Ad refers to governance of data assets feeding AI. NOT generic “data-driven decision making”.
GT4.2	L1-12	Model lifecycle governance	Role governs lifecycle of AI models from design and deployment to retraining, retirement, or updating.	Ad addresses lifecycle ownership of models.
GT4.3	L1-13	Model risk, validation & monitoring	Role accountable for validating, monitoring, testing, or assuring models in operation.	Ad mentions validation, monitoring, drift, robustness, model risk, or production assurance.
GT5 Algorithmic autonomy & opacity vs. human accountability				
GT5.1	L1-14	Human oversight & final accountability	Role ensures humans remain answerable for AI-supported or AI-automated decisions.	Ad refers to oversight, escalation, approval rights, human-in-the-loop, or decision accountability for AI outcomes.
GT5.2	L1-15	Explainability, transparency & documentation	Role makes AI systems understandable, traceable, or documentable.	Ad refers to explainability, transparency, interpretability, documentation, or audit trails of AI systems.
GT5.3	L1-16	Fairness, bias & safety protection	Role addresses harmful, unfair, unsafe, or discriminatory outcomes from AI use.	Ad mentions bias mitigation, fairness, safety, or harm prevention in relation to AI.
GT5.4	L1-17	Trustbuilding with stakeholders	Role builds interpersonal or organizational trust between stakeholders involved in AI use, inside and outside the organization.	Ad mentions trustbuilding, establishing trust, fostering trust, cultivating trust, building confidence, or building rapport in an AI-related context. NOT generic “trusted advisor” without AI context.
Layer 2: skill codes (Todd et al., 1995)				
SK1	L2-01	Technical IT/AI knowledge	Required technical competence in AI, ML, data science, cloud, engineering, tooling, or infrastructure.	Concrete technical knowledge or tool requirements related to AI.

Code (A)	ID (B)	Label	Definition	Code when
SK2	L2-02	Business/domain knowledge	Required industry, functional, or business-process knowledge to identify and shape AI use cases.	Ad asks for business acumen, domain expertise, process knowledge, or commercial understanding.
SK3	L2-03	Systems/integration knowledge	Skills for connecting AI into enterprise systems, processes, platforms, workflows, and architectures.	Ad stresses integration of AI into existing IS landscape.

A4 De-labeled prompt and change log

Prompt B is a copy of prompt A that keeps every definition, code-when rule, indicator, coding rule, and worked example word for word. It replaces code names with neutral IDs and drops theoretical headings and references. The word "governance" therefore remains where a definition or indicator uses it as content, as in "data governance".

The script `tools/build_delabeled_prompt.py` derives B from A through explicit replacements. Each replacement has an expected number of hits, and the script stops if a count differs or if a theory marker (code prefix, author reference, construct name) remains in B. Its option `--check` confirms that the shared B files are identical to a fresh derivation. Table A4 lists all 28 edits. The order of the codes stays the same.

Table A4. Edits from prompt A to prompt B

ID	Prompt A	Prompt B	Hits	Reason
S1	`apply a theory-derived codebook`	`apply a codebook`	1	reference to theoretical derivation removed
U1	`- Layer 1: Governance tension codes (GT) — derived from IS governance theory ⏎ - Layer 2: Skill/competence codes (SK) — derived from Todd et al. (1995)`	`- Layer 1: Responsibility codes (L1) ⏎ - Layer 2: Skill/competence codes (L2)`	1	construct name, theory and reference removed; layers kept as operational levels
U2	`maps to at least one GT or SK code`	`maps to at least one L1 or L2 code`	1	code prefixes neutralized
U3	`contain no AI-relevant governance or competence content`	`contain no AI-relevant responsibility or competence content`	1	'governance' as construct name replaced by the layer name
U4	`GT_Code: NONE, SK_Code: NONE`	`L1_Code: NONE, L2_Code: NONE`	2	output fields neutralized
U5	`- GT codes (Layer 1) are assigned EXCLUSIVELY`	`- L1 codes (Layer 1) are assigned EXCLUSIVELY`	1	code prefixes neutralized
U6	`- SK codes (Layer 2) are assigned EXCLUSIVELY`	`- L2 codes (Layer 2) are assigned EXCLUSIVELY`	1	code prefixes neutralized
U7	`assign GT codes only if the language`	`assign L1 codes only if the language`	1	code prefixes neutralized
U8	`## CRITICAL RULE: DUAL CODING GT + SK`	`## CRITICAL RULE: DUAL CODING L1 + L2`	1	code prefixes neutralized
U9	`A single passage may receive both GT codes and SK codes simultaneously if it describes a governance responsibility (GT layer) AND implies a required competence (SK layer).`	`A single passage may receive both L1 codes and L2 codes simultaneously if it describes a responsibility (L1 layer) AND implies a required competence (L2 layer).`	1	code prefixes neutralized; 'governance' as construct name removed

ID	Prompt A	Prompt B	Hits	Reason
U10	When a passage addresses multiple governance tensions,	When a passage addresses multiple responsibilities,	1	construct name 'governance tension' replaced; rule unchanged
U11	If one tension is clearly dominant	If one responsibility is clearly dominant	1	as U10
U12	note the secondary tension in the Rationale	note the secondary responsibility in the Rationale	1	as U10
U13	### LAYER 1 — GOVERNANCE TENSION CODES	### LAYER 1 — RESPONSIBILITY CODES (L1)	1	construct name removed
U14	### LAYER 2 — SKILL/COMPETENCE CODES (Todd et al., 1995)	### LAYER 2 — SKILL/COMPETENCE CODES (L2)	1	reference removed
U15	or platforms as a governance dimension.	or platforms.	1	theoretical framing inside one definition (GT3.1) removed; the spreadsheet version of the codebook does not contain this phrase either
U16	"GT_Code": "<semicolon-separated GT codes, or NONE>",	"L1_Code": "<semicolon-separated L1 codes, or NONE>",	1	output schema neutralized
U17	"SK_Code": "<semicolon-separated SK codes, or NONE>",	"L2_Code": "<semicolon-separated L2 codes, or NONE>",	1	output schema neutralized
U18	why GT or SK applies	why L1 or L2 applies	1	output schema neutralized
U19	Example B — Dual GT + SK passage:	Example B — Dual L1 + L2 passage:	1	example heading neutralized
U20	Responsibilities section, so no SK code applies.	Responsibilities section, so no L2 code applies.	1	example rationale neutralized
U21	"GT_Code":	"L1_Code":	3	field names in examples A to C
U22	"SK_Code":	"L2_Code":	3	field names in examples A to C
R1	group headings, e.g. GT1 — Centralization of AI decision rights vs. decentralized experimentation	removed	5	group headings GT1 to GT5 removed (theoretical grouping)
R2	code lines, e.g. GT1.1 | Enterprise AI ownership & standards	neutral ID, e.g. L1-01	17	code ID and label replaced by a neutral ID (label dropped)
R3	code lines, e.g. SK1 | Technical IT/AI knowledge	neutral ID, e.g. L2-01	3	code ID and label replaced by a neutral ID
R4	remaining code IDs in the examples, e.g. GT2.3	neutral ID, e.g. L1-06	6	remaining code IDs in the examples (code field and rationale)
R5	remaining code IDs in the examples, e.g. SK3	neutral ID, e.g. L2-03	2	remaining code IDs in the examples

SHA-256 hashes (first 12 characters) of the prompt files are A_system `db42ac69f249`, A_user `8b947b52014c`, B_system `198dd6706dde`, and B_user `073fc29c5590`. `prompts/case/change_log_A_to_B.md` contains the full line-by-line difference.

A5 Model, settings, and output checks

Table A5. Settings of the first run and of the planned runs

Item	First run	Phase 1 runs A, A', B and Phase 2
Model	Claude Sonnet 4.6 (`claude-sonnet-4-6`) via the Anthropic API	one current model, the same for runs A, A', and B
Documents per call	one	one
Maximum output tokens	4,000	8,000, with one retry at twice the limit if a response stops at the limit
Temperature	no parameter sent	no parameter sent
Pause between calls	1.5 seconds	1.5 seconds
Raw responses	parsed and stored per segment	archived unchanged per document (JSONL) with model ID, prompt hash, stop reason, and tokens
Failed responses	re-run (88 of 794)	logged by document and role, re-run with every attempt archived, reported by role
Output checks	human review of LOW segments and inductive suggestions	codes outside the codebook flagged, quoted segments checked against the source text

In the planned runs, `tools/run_coding.py` stops at once on structural API errors (HTTP 400, 401, 403, 404) and retries transient ones. A quoted segment counts as verbatim if it appears in the source text after whitespace normalization. Runs A, A', and B use the same input slot and run close in time, so that model updates are unlikely to explain differences between them.

A6 Samples and seeds

All random draws use the seed 20261101 and are made with `tools/draw_samples.py`. Table A6.1 lists the samples.

Table A6.1. Samples

Sample	Population	Size	Draw	File
Confidence-calibration test	6,773 coded segments of the first run, calibration ads excluded	227	stratified, without replacement (Table A6.2)	`case/samples/calibration_test_sample.csv`
Adjudication, theory-label test	ad-code cells in which A and A' agree and B departs	50	simple random	drawn after the runs
Recall check	ads without any code in the first run	86	all	`case/samples/recall_check_uncoded_ads.csv`
Document-level coding, Phase 2	second corpus	about 170 (10%)	simple random	drawn after cleaning

Table A6.2. Strata of the confidence-calibration sample

Stratum	Definition	Population	Sample
LOW	model confidence LOW	27	27 (all)
MEDIUM, contextual	MEDIUM, rationale cites contextual specificity	1,238	50
MEDIUM, lexical	MEDIUM, AI named in the passage	347	50
HIGH	model confidence HIGH	5,161	100
Total		6,773	227

The MEDIUM level is split because the eligibility rule demands MEDIUM or LOW confidence for contextual coding (A13). Comparing both strata shows whether MEDIUM records uncertainty or only the rule. A segment counts as contextual if its rationale contains the word "contextual".

The coders work in blind sheets made by `tools/make_blind_sheets.py`. For each segment they see the text, the 300 characters before it, a guess of its section heading, and a link to the ad. They do not see the model's codes, confidence, rationale, or stratum, and the order of segments is random. The adjudication sheets show the ad and one code with its definition, without the prompt that assigned it.

A7 Agreement measures

For two binary ratings of the same units, percent agreement p_o is the share of units rated alike. Cohen's kappa corrects it for chance agreement p_e, computed from the shares a and b of positive ratings by each rater:

$$\kappa = (p_o - p_e) / (1 - p_e), \text{ where } p_e = a\,b + (1 - a)(1 - b)$$

Because kappa falls for rare codes even when agreement is high, we also report the prevalence- and bias-adjusted kappa (Byrt et al., 1993):

$$\text{PABAK} = 2\,p_o - 1$$

Table A7. Units of agreement

Test	Unit	Agreement
Theory-label test	ad × code, 1 if the code appears in at least one segment of the ad (794 × 20 cells per run)	κ, PABAK, and prevalence per code
Confidence-calibration test	segment	percent agreement; a segment agrees only if its full set of codes matches
Recall check, Phase 2	ad × code	percent agreement, κ, and recall per code

Bootstrap intervals are percentile intervals from 2,000 resamples. In the theory-label test we resample ads, not cells, because the cells of one ad are not independent. In the confidence-calibration test we resample segments within strata.

A8 Decision rule of the theory-label test

The thresholds were fixed on October 19, 2026, before any Phase 1 run and before any blind coding (`case/preregistration/thresholds.md`). A code shows a label effect only if both conditions hold:

1. $\kappa(A, A') - \kappa(A, B) > 0.10$, and the 95% bootstrap interval of this difference excludes zero.

2. The prevalence difference B − A lies outside the 95% bootstrap interval of the prevalence difference A′ − A, the noise band of two identical runs.

The substantive conclusion holds if every code whose prevalence gap between AI leadership roles and comparison roles is at least 10 percentage points in run A keeps the sign of its gap in run B. If only single codes shift, sharper definitions suffice. If the conclusion changes, we recommend coding with a de-labeled prompt.

For the adjudication, both authors judge each of the 50 cells blind, without knowing which prompt assigned the code, and record whether the text supports the code and which passage does so.

A9 Review-effort curve

For each confidence level l (LOW, MEDIUM, HIGH), let p_l be the percent agreement between model and reference coding, h_l the percent agreement between the two coders before adjudication, and w_l the share of level l among all coded segments. Review proceeds from LOW to HIGH, in random order within a level. With s the share of all segments reviewed, W_l the cumulative share of the levels up to l, and $r_l(s)$ the reviewed fraction of level l:

$$r_l(s) = \min(1, \max(0, (s - W_{l-1}) / w_l))$$

$$PA(s) = \Sigma_l \, w_l \, [\, r_l(s) \, h_l + (1 - r_l(s)) \, p_l \,]$$

Random order within a level makes the expected curve linear inside each level, so $PA(s)$ is piecewise linear with kinks at s = 0.4% and s = 23.5%. The review share to plan for is the smallest s with $PA(s) \geq 0.90$, reported with its bootstrap interval. The MEDIUM value p_{MEDIUM} pools the two MEDIUM strata with their population weights (1,238 and 347).

Table A9. Weights of the confidence levels

Level	Coded segments, first run	Weight w_l	Without calibration ads
LOW	27	0.4%	0.4%
MEDIUM	1,647	23.1%	23.4%
HIGH	5,452	76.5%	76.2%
Total	7,126	100%	100%

We use the weights of the full first run, as fixed in the preregistration. Both sets of weights differ by less than half a percentage point. The 90% target follows Lombard et al. (2002), who consider intercoder agreement of .90 or greater nearly always acceptable. Figure 2 of the proposal uses assumed values (82% agreement without review, 97% agreement between coders) and smooth functional forms. It shows possible shapes, not results.

A10 Precision of the estimates

Table A10 gives the half-width of a 95% confidence interval for a proportion p estimated from n units, $1.96 \sqrt{(p(1 - p) / n)}$. The worst case p = 0.5 gives the widest interval.

Table A10. Half-widths of 95% confidence intervals

n	Used for	Half-width at p = 0.5	Half-width at p = 0.9
27	LOW segments	±19 pp	±11 pp
50	adjudicated cells, each MEDIUM stratum	±14 pp	±8 pp
86	recall check	±11 pp	±6 pp
100	HIGH segments	±10 pp	±6 pp
170	document-level coding, Phase 2	±7.5 pp	±4.5 pp

The LOW estimate is the least precise, but LOW carries a weight of 0.4%, so its error moves the curve by less than 0.1 percentage points. The HIGH estimate matters most, because HIGH carries three quarters of the weight.

A11 Recall check of uncoded ads

The confidence-calibration sample contains only passages the model extracted, so it cannot show what the model missed. The first run left 86 of 794 ads without any code (Table A11). Both authors read each of these ads in full and code it independently with the full codebook in a blind sheet (`tools/make_blind_sheets.py documents`). They then resolve disagreements. A missed code is a code the resolved coding assigns. We report the share of the 86 ads with at least one missed code, by role group, and assign each miss an error mode (A12).

Table A11. Ads without any code in the first run

Role group	Roles	Ads without code	Share of the group's ads
AI leadership	AI Manager 4, Head of AI 3, Chief AI Officer 0, AI Lead 1	8	1.3%
Comparison	CTO 32, CIO 37, CDO 9	78	42.2%
Total		86	10.8%

A12 Error modes

We sort the errors that both tests reveal into error modes. Table A12 defines the four modes we start with. Three are suggested by first-run rationales, and theory filling is the mode the theory-label test probes. Both authors independently assign one mode to every error, may add modes the data show, and resolve disagreements. We report the frequencies of the modes by confidence level and prompt condition, and the agreement between the authors on the assignment.

Table A12. Initial error modes

Mode	Definition
Over-extraction from context	The model codes a passage that does not refer to AI, based on the role or a general section rather than a clearly AI-dedicated section.
Definition stretching	The passage refers to AI, but its content does not meet the definition or the code-when rule of the assigned code.
Cue-word coding	The model assigns a code because an indicator word appears, although the passage does not carry the meaning the code requires.
Theory filling	The model assigns a code that the theoretical framing leads it to expect for the role, without support in the text.

A13 First-run descriptives

The first run returned 7,294 rows, of which 7,126 are coded segments and 168 record ads or passages without any code. Table A13.1 covers all 794 ads. The model proposed an inductive label for 118 segments in 61 ads, with 117 distinct labels. We pooled all suggestions and kept none as a new code after a relevancy and frequency analysis.

Table A13.1. Output by role

Role	Ads	Coded segments	Segments per ad	Failed first-pass responses	Ads without any code
AI Manager	154	1,484	9.6	16 (10%)	4 (3%)
Head of AI	298	3,441	11.5	48 (16%)	3 (1%)
Chief AI Officer	55	646	11.7	9 (16%)	0 (0%)
AI Lead	102	1,062	10.4	13 (13%)	1 (1%)
CTO	89	304	3.4	1 (1%)	32 (36%)
CIO	79	154	1.9	1 (1%)	37 (47%)
CDO	17	35	2.1	0 (0%)	9 (53%)
Total	794	7,126	9.0	88 (11%)	86 (11%)

Failed responses were JSON that could not be parsed. Of the 88, 42 were cut off, as is typical when output reaches the token limit, and 46 were otherwise malformed. All 88 were re-run. Because 86 of them came from AI leadership roles, which yield three to six times more segments per ad, dropping them would have biased every comparison between role groups.

Table A13.2. Confidence levels

Confidence	Coded segments	Share	Rationale cites contextual specificity
HIGH	5,452	76.5%	
MEDIUM	1,647	23.1%	1,289 (78%)
LOW	27	0.4%	
Total	7,126	100%	

The prior study compared the model's codes for the 32 calibration ads with our consensus coding, using the same prompt as the main run, including codebook, coding rules, and worked examples. Percent agreement was 98.5% and Cohen's κ was 0.880 across 277 code segments, which is almost perfect agreement in the terms of Landis and Koch (1977). Our consensus coding is in `case/data/calibration_coding.csv`.

A14 Second corpus for Phase 2

Phase 2 runs the revised workflow on a second corpus of job ads from LinkedIn and Adzuna. It covers the four AI leadership roles and six comparison roles. The comparison roles now span the top and the middle management tier, as the AI leadership roles do, in three domains (Table A14).

Table A14. Second corpus after cleaning

Role	Tier	Domain	Ads
Chief AI Officer	top	AI	38
Head of AI	middle	AI	243
AI Manager	middle	AI	204
AI Lead	middle	AI	206
CTO	top	product and engineering	166
CIO	top	information systems	185
Chief Digital and Data Officer	top	digital and data	44
Head of Engineering	middle	product and engineering	195
Head of IT	middle	information systems	201
Head of Digital and Data	middle	digital and data	202
Total			1,684

Of 2,007 collected ads, we removed 84 whose full text was cut off and could not be retrieved again, 55 repeated postings of the same job in 41 groups (keeping the longest text of each group), and 184 texts under 1,500 characters, which were machine-generated platform summaries rather than the ads themselves. The removed summaries had a median of 514 characters, whereas the remaining ads have a median of about 6,050 characters. Thresholds between 800 and 2,000 characters would leave between 1,724 and 1,631 ads, so the exact threshold matters little. Of the 1,684 ads, 1,002 come from LinkedIn and 682 from Adzuna, and 1,600 are in English and 84 in German. Before coding, we repeat the integrity checks of `guide/01_corpus.md`, so the final number may differ slightly.

A15 Costs

The provider invoiced about USD 50 for the entire first run, including repeated calls, which is about USD 0.06 per ad. The first pass of the 706 ads with valid output used 3,492,827 input and 1,331,896 output tokens, a mean of 4,947 input and 1,887 output tokens per ad.

At the cost per ad of the first run, the three Phase 1 runs of 794 ads would cost about USD 150, and one Phase 2 run of 1,684 ads about USD 106. Run stability in Phase 2 requires at least two runs per ad. Actual costs depend on the model and its prices at the time of the runs, and the higher token limit may raise them. `tools/estimate_cost.py` estimates costs from token counts and current prices, or from an earlier invoice.

A16 Data, ethics, and API terms

The job ads are public postings from LinkedIn and Adzuna. They describe roles, not persons. The repository shares, for each ad, an ID, the role, the source platform, the URL without tracking parameters, the posting month, the language, the text length, and a hash of the full text, together with the coded segments, the raw model outputs, our calibration coding, and the samples. It does not share full ad texts, which platform terms may protect. Coded segments are short verbatim passages needed to check the coding. We scanned all shared text fields for e-mail addresses, phone numbers, and profile links and found none. The study involves no human participants other than the authors, who serve as coders.

According to Anthropic's privacy information, inputs and outputs of its commercial API are not used to train models by default (Anthropic, n.d.). Code in the repository is licensed under MIT, and documentation,

templates, and our data under CC BY 4.0 (`LICENSE`, `LICENSE-docs.md`). Quoted passages from job ads remain subject to the rights of their authors and the terms of the platforms.

A17 Full coding prompt

Prompt A below is the coding prompt of the illustrative case and the original prompt of the theory-label test. Text in braces, such as `{ad_id}`, is filled for each ad. The prompt ends with the full text of the ad.

System prompt

```
You are a research assistant supporting a scientific content analysis of job
advertisements for AI leadership roles and technology leadership roles. Your task
is to apply a theory-derived codebook to job ad texts and return structured coding
decisions.

You operate with precision and consistency. You follow the coding rules exactly as
specified. You do not interpret beyond what the text says. You do not add codes
that are not supported by explicit textual evidence. You always return valid JSON.
```

User message

```
## RESEARCH CONTEXT

You are assisting with an academic content analysis study examining how
organizations configure dedicated AI leadership roles (AI Manager, Head of AI,
Chief AI Officer, AI Lead) compared to traditional technology leadership roles
(CTO, CIO, CDO).

The analysis uses a two-layer codebook:
- Layer 1: Governance tension codes (GT) — derived from IS governance theory
- Layer 2: Skill/competence codes (SK) — derived from Todd et al. (1995)

Your task is to read a complete job advertisement, identify all passages that are
relevant for coding, and return a structured coding output for each identified
passage.

---

## YOUR TASK

Read the full job advertisement provided below. Identify every passage — a
sentence, a group of sentences, or a bullet point cluster — that meets the AI
specificity requirement and maps to at least one code in the codebook. Extract
each such passage as a text segment and assign the appropriate codes.

A passage is worth extracting when it contains a meaningful reference to AI, ML,
GenAI, or direct equivalents AND maps to at least one GT or SK code. Do not
extract passages that are purely administrative (salary, benefits, equal
opportunity statements, company boilerplate) or that contain no AI-relevant
governance or competence content.

If the entire job advertisement contains no AI-specific content, return a single
entry with GT_Code: NONE, SK_Code: NONE, and explain in the Rationale why no codes
apply.

---

## CRITICAL RULE: AI SPECIFICITY REQUIREMENT

Before coding any passage, check whether it meets the AI specificity requirement.
```

A passage is eligible for coding ONLY if it contains a meaningful reference to AI, ML, GenAI, large language models, or direct equivalents (e.g., machine learning, artificial intelligence, agentic AI).

Two types of AI specificity are recognized:

1. LEXICAL SPECIFICITY: AI, ML, GenAI, or a direct equivalent is explicitly named within the passage itself. This is the default and preferred basis for coding.

2. CONTEXTUAL SPECIFICITY: AI is not named in the passage, but the passage is embedded within a clearly AI-dedicated section of the job ad (e.g., a section titled "AI Governance" or "AI Strategy") and the passage cannot be meaningfully read as referring to anything other than AI. Contextual specificity does NOT apply simply because the overall role is AI-related. When coding on the basis of contextual specificity, confidence must be set to Medium or Low.

If a passage has NO AI specificity, do not extract it. The absence of AI content is itself a finding — capture it in the Rationale of the NONE entry if the entire ad has no AI content.

CRITICAL RULE: RESPONSIBILITIES VS. REQUIREMENTS

- GT codes (Layer 1) are assigned EXCLUSIVELY to passages from responsibilities, tasks, mandates, or accountabilities sections.
- SK codes (Layer 2) are assigned EXCLUSIVELY to passages from requirements, qualifications, or experience sections.
- When a passage is ambiguous, assign GT codes only if the language clearly implies organizational accountability rather than personal background.

CRITICAL RULE: DUAL CODING GT + SK

A single passage may receive both GT codes and SK codes simultaneously if it describes a governance responsibility (GT layer) AND implies a required competence (SK layer). Assign both when both dimensions are clearly present in the same passage.

CRITICAL RULE: PRIMARY CODE PRIORITIZATION

When a passage addresses multiple governance tensions, assign all codes that are clearly and substantively present. If one tension is clearly dominant and others are only marginally implied, assign only the dominant code and note the secondary tension in the Rationale. The threshold: would a trained human coder independently identify this code from the passage alone? If yes, assign it. If no, note it in the Rationale only.

CODEBOOK

LAYER 1 — GOVERNANCE TENSION CODES

Apply to: responsibilities / tasks / mandates sections only.

GT1 — Centralization of AI decision rights vs. decentralized experimentation

GT1.1 | Enterprise AI ownership & standards
Definition: Role given org-wide authority to define AI priorities, standards, policies, or decision rights.
Code when: Ad assigns enterprise-level ownership, standard-setting, or governance authority. NOT simple collaboration without authority.
Indicators: "own enterprise AI strategy", "define AI standards", "set AI policies", "enterprise AI operating model"

GT1.2 | Federated coordination across BUs
Definition: Role coordinates distributed AI across functions/BUs while preserving local autonomy.
Code when: Role integrates, aligns, or harmonizes AI work across units. Code CoE/hub-and-spoke. NOT if confined to one team.
Indicators: "coordinate AI across BUs", "center of excellence", "hub and spoke", "align central and local AI", "cross-functional team"

GT1.3 | Local experimentation enablement
Definition: Role enables BU-level experimentation, pilots, sandboxes, or decentralized AI use-case development.
Code when: Role explicitly supports local experimentation or bottom-up AI. NOT generic "innovation" alone.
Indicators: "enable experimentation", "support AI pilots", "incubate AI use cases", "sandbox", "empower product teams"

GT2 — Innovation and generativity vs. control, risk, and compliance

GT2.1 | AI opportunity creation & experimentation
Definition: Role identifies, initiates, or scales AI opportunities for innovation and value creation.
Code when: Ad emphasizes discovery, experimentation, use-case ID, or growth through AI. Exclude pure compliance tasks.
Indicators: "identify AI opportunities", "drive AI innovation", "build AI roadmap", "pilot and scale", "unlock business value", "AI portfolio"

GT2.2 | Responsible AI, risk & compliance
Definition: Role addresses ethical, legal, regulatory, privacy, or risk issues related to AI.
Code when: Ad mentions responsible AI, ethics, regulation, legal compliance, or risk mitigation related to AI.
Indicators: "responsible AI", "AI ethics", "regulatory compliance", "AI risk management", "privacy and security", "ethical AI framework"

GT2.3 | Governance frameworks & controls
Definition: Role designs or maintains formal governance mechanisms for AI oversight and control.
Code when: Role builds frameworks, committees, controls, review processes, or policies. NOT vague "best practices".
Indicators: "establish AI governance framework", "create review board", "define controls", "AI policy framework", "guidelines", "guardrails"

GT2.4 | AI enablement & change management
Definition: Role drives organizational AI adoption by managing the human and cultural side — enablement programs, internal champions, stakeholder engagement, friction reduction.
Code when: Ad assigns explicit responsibility for change management, adoption acceleration, workforce enablement, or internal advocacy related to AI. NOT generic "collaboration" without adoption mandate.
Indicators: "enablement", "AI literacy", "capacity building", "training", "change management", "champions", "ambassadors", "AI culture", "reduce friction"

GT3 — Technical and model-specific expertise vs. enterprise strategic alignment

GT3.1 | Deep AI/ML/GenAI expertise
Definition: Role requires substantial technical knowledge of AI methods, models, tools, or platforms as a governance dimension.
Code when: Ad asks for ML, GenAI, LLMs, data science, AI architecture, or tooling depth. NOT basic digital literacy.
Indicators: "machine learning", "LLMs", "deep learning", "AI architecture", "MLOps", "data science leadership"

GT3.2 | AI strategy & business value alignment
Definition: Role aligns AI with business strategy, competitive priorities, or value creation.
Code when: Ad links AI to enterprise strategy, transformation, or ROI. NOT generic leadership language without strategic alignment.
Indicators: "define AI strategy", "align AI with business goals", "value realization", "AI transformation", "strategic roadmap", "measurable value"

GT3.3 | Business-technical translation
Definition: Role translates between business needs and AI capabilities, or between executives and technical teams.
Code when: Ad explicitly frames role as interpreter, connector, or translator across business and technical domains.
Indicators: "translate business problems into AI", "bridge technical and nontechnical stakeholders", "communicate AI to executives"

GT4 — Data governance vs. model governance & lifecycle accountability

GT4.1 | Data governance responsibility
Definition: Role responsible for data quality, access, lineage, stewardship, privacy, or governance of inputs to AI.
Code when: Ad refers to governance of data assets feeding AI. NOT generic "data-driven decision making".
Indicators: "data governance", "data quality", "data stewardship", "data access controls", "data lineage", "trusted data"

GT4.2 | Model lifecycle governance
Definition: Role governs lifecycle of AI models from design and deployment to retraining, retirement, or updating.
Code when: Ad addresses lifecycle ownership of models.
Indicators: "model lifecycle management", "model deployment", "model retraining", "retire or update models"

GT4.3 | Model risk, validation & monitoring
Definition: Role accountable for validating, monitoring, testing, or assuring models in operation.
Code when: Ad mentions validation, monitoring, drift, robustness, model risk, or production assurance.
Indicators: "model validation", "performance monitoring", "model drift", "AI assurance", "model risk management", "risk assessment", "evaluation metrics"

GT5 — Algorithmic autonomy & opacity vs. human accountability

GT5.1 | Human oversight & final accountability
Definition: Role ensures humans remain answerable for AI-supported or AI-automated decisions.
Code when: Ad refers to oversight, escalation, approval rights, human-in-the-loop, or decision accountability for AI outcomes.
Indicators: "human oversight", "final decision authority", "escalation

procedures", "human-in-the-loop"

GT5.2 | Explainability, transparency & documentation
Definition: Role makes AI systems understandable, traceable, or documentable.
Code when: Ad refers to explainability, transparency, interpretability, documentation, or audit trails of AI systems.
Indicators: "explainable AI", "transparency", "document model decisions", "traceability", "audit trail"

GT5.3 | Fairness, bias & safety protection
Definition: Role addresses harmful, unfair, unsafe, or discriminatory outcomes from AI use.
Code when: Ad mentions bias mitigation, fairness, safety, or harm prevention in relation to AI.
Indicators: "bias detection", "fair AI", "safe AI deployment", "trustworthy AI", "harm mitigation"

GT5.4 | Trustbuilding with stakeholders
Definition: Role builds interpersonal or organizational trust between stakeholders involved in AI use, inside and outside the organization.
Code when: Ad mentions trustbuilding, establishing trust, fostering trust, cultivating trust, building confidence, or building rapport in an AI-related context. NOT generic "trusted advisor" without AI context.
Indicators: "stakeholder trust building", "trusted advisor" (AI context only), "establish credibility", "build confidence in AI"

LAYER 2 — SKILL/COMPETENCE CODES (Todd et al., 1995)

Apply to: requirements / qualifications / experience sections only.

SK1 | Technical IT/AI knowledge
Definition: Required technical competence in AI, ML, data science, cloud, engineering, tooling, or infrastructure.
Code when: Concrete technical knowledge or tool requirements related to AI.
Indicators: "Python", "cloud AI platforms", "data engineering", "MLOps stack", "AI architecture", "LLMs", "machine learning"

SK2 | Business/domain knowledge
Definition: Required industry, functional, or business-process knowledge to identify and shape AI use cases.
Code when: Ad asks for business acumen, domain expertise, process knowledge, or commercial understanding.
Indicators: "strong business acumen", "industry expertise", "understanding of operations", "commercial orientation", "domain knowledge"

SK3 | Systems/integration knowledge
Definition: Skills for connecting AI into enterprise systems, processes, platforms, workflows, and architectures.
Code when: Ad stresses integration of AI into existing IS landscape.
Indicators: "enterprise architecture", "systems integration", "platform integration", "connect AI into core systems"

OUTPUT FORMAT

Return a JSON array. Each element represents one coded passage. Use this structure:

{

```
  "Seg_ID": "<Ad_ID>_S<sequential number starting at 01>",
  "Ad_ID": "<provided Ad_ID>",
  "Role_Family": "<provided Role_Family>",
  "Text_Segment": "<exact extracted text from the job ad>",
  "GT_Code": "<semicolon-separated GT codes, or NONE>",
  "SK_Code": "<semicolon-separated SK codes, or NONE>",
  "Inductive_Code": "<short free-text label if new pattern, else null>",
  "Confidence": "<HIGH / MEDIUM / LOW>",
  "Rationale": "<2-4 sentences: which words triggered which codes, why GT or SK
applies, and if contextual specificity was used>"
}

Important output rules:
- Return ONLY a valid JSON array. No preamble, no explanation outside the JSON.
- Generate Seg_IDs sequentially per ad: {ad_id}_S01, {ad_id}_S02 etc.
- If the entire ad has no AI-relevant content, return a single element with
GT_Code: NONE, SK_Code: NONE, Text_Segment: null, and a Rationale explaining the
absence.
- Always write the Rationale in English, even for German-language ads.
- Extract Text_Segment verbatim from the ad — do not paraphrase or summarize.
- Do not extract boilerplate: salary, benefits, equal opportunity statements,
application instructions.

---

## EXAMPLES

Example A — Single responsibility passage:
Input contains: "Establish procedures and frameworks for responsible AI monitoring
and testing."
Output element:
{"Seg_ID": "JOB_0004_S01", "Ad_ID": "JOB_0004", "Role_Family": "AI Manager",
"Text_Segment": "Establish procedures and frameworks for responsible AI monitoring
and testing.", "GT_Code": "GT2.3", "SK_Code": "NONE", "Inductive_Code": null,
"Confidence": "HIGH", "Rationale": "The passage assigns responsibility for
building formal governance mechanisms — 'procedures and frameworks' — for AI
oversight, mapping to GT2.3. 'Responsible AI monitoring and testing' provides
lexical AI specificity. Responsibilities section, so no SK code applies."}

Example B — Dual GT + SK passage:
Input contains: "Design and deliver AI solutions aligned to business priorities,
enterprise architecture, and trusted data."
Output element:
{"Seg_ID": "JOB_0637_S03", "Ad_ID": "JOB_0637", "Role_Family": "AI Lead",
"Text_Segment": "Design and deliver AI solutions aligned to business priorities,
enterprise architecture, and trusted data.", "GT_Code": "GT3.2; GT4.1", "SK_Code":
"SK3", "Inductive_Code": null, "Confidence": "HIGH", "Rationale": "GT3.2: 'aligned
to business priorities' links AI delivery to enterprise strategy. GT4.1: 'trusted
data' signals data governance as input to AI. SK3: 'enterprise architecture'
implies systems/integration knowledge. The passage describes a responsibility that
simultaneously implies a required competence, justifying dual coding."}

Example C — Zero codes:
Output: [{"Seg_ID": "JOB_0005_S01", "Ad_ID": "JOB_0005", "Role_Family": "CTO",
"Text_Segment": null, "GT_Code": "NONE", "SK_Code": "NONE", "Inductive_Code":
null, "Confidence": "HIGH", "Rationale": "No AI-specific content found in the
entire job ad. The posting describes general software engineering leadership
without any reference to AI, ML, GenAI, or direct equivalents. Zero codes
assigned."}]
```

```
---

## INPUT

Ad_ID: {ad_id}
Role_Family: {role_family}
Language: {language}

Full job advertisement text:
{full_text}
```

References

Anthropic. (n.d.). *Is my data used for model training?* Anthropic Privacy Center. Retrieved October 19, 2026, from https://privacy.claude.com/en/articles/7996868-is-my-data-used-for-training

[Blinded for review]. (forthcoming). [Prior study on the configuration of AI leadership roles, accepted at a peer-reviewed IS conference].

Byrt, T., Bishop, J., & Carlin, J. B. (1993). Bias, prevalence and kappa. *Journal of Clinical Epidemiology, 46*(5), 423–429. https://doi.org/10.1016/0895-4356(93)90018-V

Landis, J. R., & Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics, 33*(1), 159–174. https://doi.org/10.2307/2529310

Lombard, M., Snyder-Duch, J., & Bracken, C. C. (2002). Content analysis in mass communication: Assessment and reporting of intercoder reliability. *Human Communication Research, 28*(4), 587–604. https://doi.org/10.1093/hcr/28.4.587

Todd, P. A., McKeen, J. D., & Gallupe, R. B. (1995). The evolution of IS job skills: A content analysis of IS job advertisements from 1970 to 1990. *MIS Quarterly, 19*(1), 1–27. https://doi.org/10.2307/249709